

Singletons vs. class (un)loading

Find out how Java 1.2 changes the way we load and unload classes in Java

Have you written any classes adhering to the Singleton pattern? Have you run into odd bugs where your singleton objects just seem to vanish? Are you confused about when classes can be reloaded? If you answered yes to these questions, you're reading the right article!

Coders trying to implement and test the Singleton pattern (see [Resources](#)) in Java 1.1 often complained that their singleton classes were getting garbage collected; that is, the runtime unloaded the classes right out from under them! Fortunately, the upcoming Java 1.2 specification clarifies the rules for loading and unloading classes, making writing singletons much simpler.

This tip explores implementing singletons in Java and describes what we need to do to make them work in the various versions of Java floating around.

Singletons defined

A *singleton* is a class for which only one instance can exist within a program. In other words, only one, *single* object of that particular class can be created in a program.

Here's a simple, quick-and-dirty singleton class:

```
public class SimpleSingleton
{
    // Create the single instance, make it available statically, and
    // don't let it be redefined.

    private static final SimpleSingleton instance = new SimpleSingleton();
    // Allow subclasses to override the constructor, if necessary.

    protected SimpleSingleton()
    {
        // Whatever...
    }

    // Accessor only.

    public static SimpleSingleton getInstance()
    {
        return instance;
    }

    // Methods on the object to actually do something useful.

    public void doSomething()
    {
        // Whatever.
    }
}
```

All we ever need to do to actually use the various methods of the SimpleSingleton class is to access the static instance member and use it to invoke our method of choice, like this:

```
SimpleSingleton.getInstance().doSomething();
```

With such a class, we never *need* to create and store the SimpleSingleton instance object anywhere, or worry about passing the object reference around.

Singletons in Java 1.0 and 1.1

All Java specifications require us to keep a reference to the singleton object, or lose it to the garbage collector. But, in

pre-1.2 Java specifications, the garbage collector was not only free to collect the singleton *object* but also the *SimpleSingleton class*!. This garbage collection of an actual class is also known as "class unloading."

Actually, in Java 1.0.x class unloading wasn't implemented, so our *SimpleSingleton* implementation would work just fine. Unfortunately, in Java 1.1.x even *system* classes were allowed to be unloaded. Typical workarounds for this inadvertent unloading of the singleton class include:

- Keeping a reference to the singleton as a local variable in your `main()` method (or the `run()` method for thread classes).

- Keeping a reference to the singleton as an instance variable in the class in which you define your `main()` method (or the `run()` method of a thread class).

Although these techniques work, they are problematic if you use third-party libraries and can't get to the singleton (as that particular class might be an internal helper class).

Singletons in Java 1.2

After a good bit of debate on the 'Net, combined with a number of complaints, the powers that be at Sun decided to clear up the problem by slightly modifying the Java language specification.

The new rules for determining when a class may or may not be unloaded are wonderfully simple:

- All classes loaded through the local, system classloader will never be unloaded.

- All classes loaded through other classloaders will be unloaded only when the classloader is unloaded.

So, for the vast majority of developers, this remedy gets rid of a relatively subtle, but annoying problem -- and we never have to lift a finger!

Dynamic loading and unloading of classes

For those of you who need to deal with loading and unloading classes on the fly, these new rules makes life a little clearer. Just load classes that you wish to later reload in your own classloader. When you want to reload the classes, nuke the classloader and then load the class through a newly created classloader.

Alas, there is not, at this time, any way to use classloaders in a secure environment from within an untrusted applet (as an untrusted applet is prevented from implementing classloaders). A RFE (um, er, request for enhancement) has been filed with Sun to create a constrained, untrusted classloader for use in untrusted applets.